

TPU FlexAttention: Flexible Attention Kernels on TPUs with JAX + Pallas

Kellen Kanarios¹ and Venkatkrishnan Iyer¹

¹University of Michigan

Your Algorithm Must be *Hardware Aware*

Fast attention is not just the formula

$$\text{softmax}\left(\frac{QK^{\top}}{\sqrt{d}}\right)V.$$

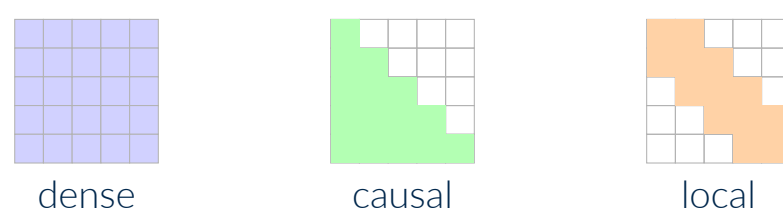
It is a **hardware schedule**: avoid materializing the full score matrix, fuse kernels, and keep data moving through on-chip memory.

- **Kernel fusion**: compute attention in one fused pipeline instead of many separate ops.
- **Online softmax**: normalize block-by-block without storing all scores.
- **Memory movement dominates**: the best kernel is the one that reduces HBM traffic.

Hardware-Aware $\oplus$ Generality

This is the tension behind FlexAttention.

- **Hardware-aware kernels** want regular tiles, predictable memory access, and fixed schedules.
- **Flexible attention variants** introduce irregular masking, local structure, and custom score changes.
- A modified attention rule may be *easy to write*, but it does *not* automatically lower to an optimal kernel.



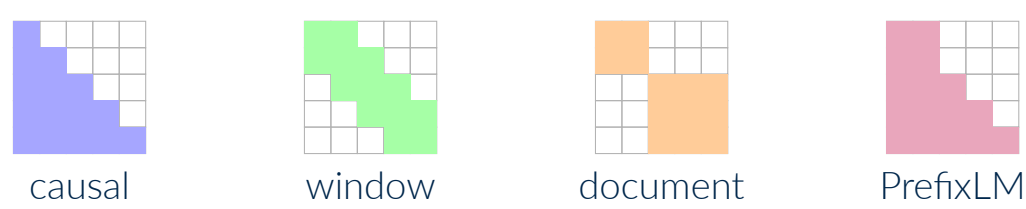
Key issue: general compilers see different semantics, but the TPU still needs one efficient execution schedule.

FlexAttention: The Abstraction

FlexAttention separates attention changes into two simple interfaces:

$$\text{Attn}(Q, K, V) = \text{softmax}\left(\frac{QK^{\top}}{\sqrt{d}} + \Delta_{\text{score}} + \Delta_{\text{mask}}\right)V$$

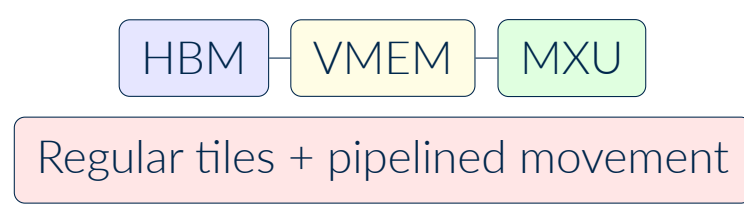
- **mask_mod**: decides which query-key pairs are allowed.
- **score_mod**: changes the score of allowed pairs.
- **Why we need both**:
 - masks give *semantic sparsity* and let the kernel skip work,
 - score mods keep the same sparsity pattern but change relative weights.



GPU $\neq$ TPU

The GPU FlexAttention paper relies on Triton/CUDA templates. TPU needs a different backend story.

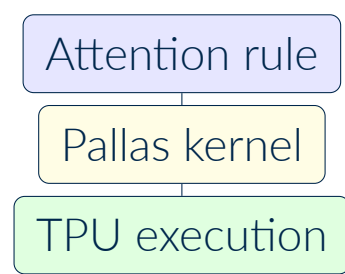
- **GPU**: Triton templates target CUDA execution and GPU-specific scheduling.
- **TPU**: performance depends on explicit tiling, prefetch, VMEM usage, and pipelining.
- **Same abstraction, different lowering**: the frontend idea transfers; the kernel schedule does not.



Pallas

Pallas is the right level of control for TPU kernel experiments.

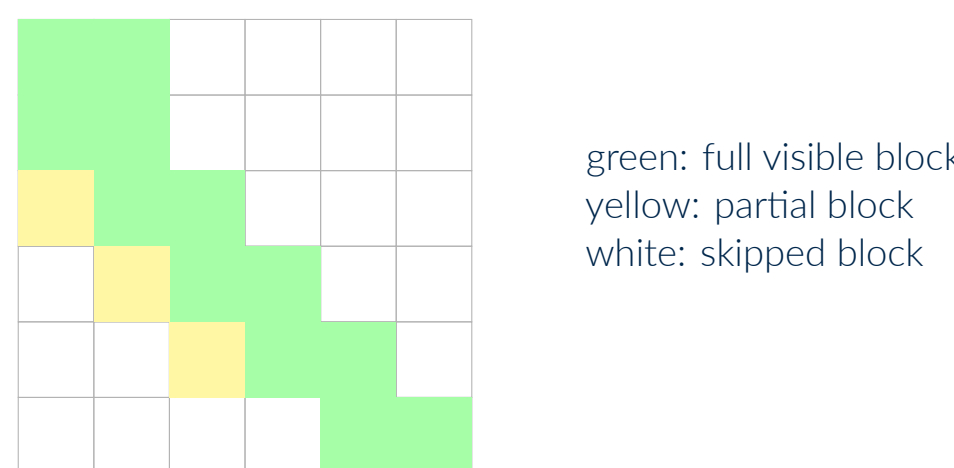
- explicit memory management
- block-level kernel structure
- scalar prefetch / asynchronous movement
- enough control to express hardware-aware schedules



Splash Attention

Splash Attention gives a strong **TPU-native backend** for sparse masking.

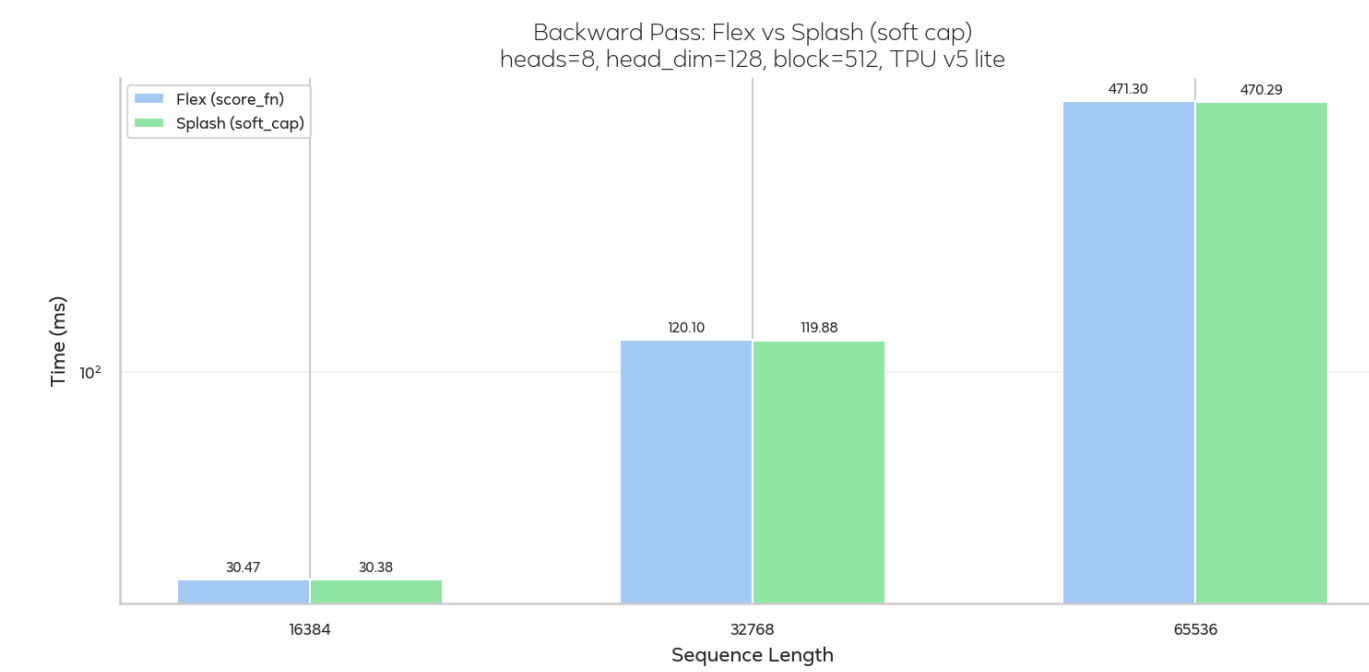
- Pallas-based fused attention kernel
- exploits **block sparsity**: skip fully masked blocks
- strong support for mask structure
- but not yet the full FlexAttention frontend:
 - easy mask patterns, yes
 - general score modification lowering, not yet
 - dense / sparse / paged still feel like separate backends



TPU FlexAttention

We implemented a flexible **score_mod** path on TPU and compared it against Splash and a naive JAX baseline on **soft-cap attention**.

- **Main result**: Flex matches Splash closely while preserving a more flexible frontend.
- **Compared with naive JAX**: the gap grows with sequence length, so fusion matters more at long context.

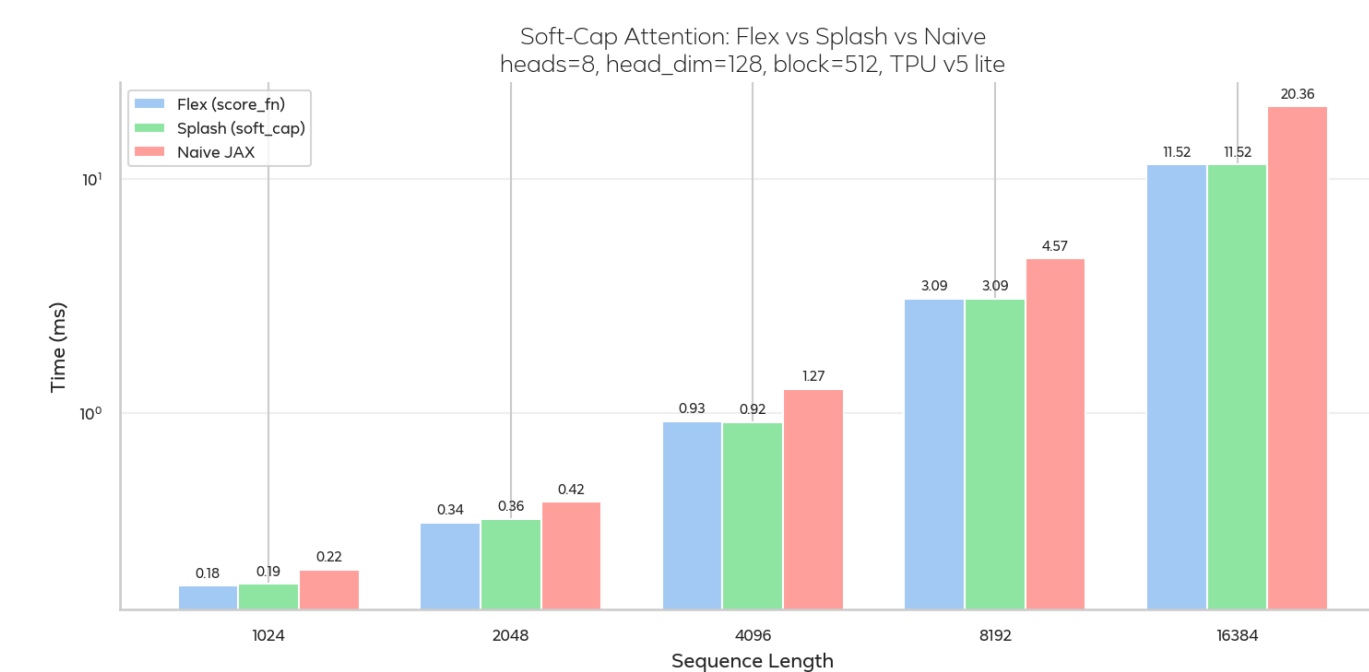


Forward soft-cap attention. Flex and Splash are nearly identical across all tested lengths: 0.18 vs 0.19 ms at 1k, 0.93 vs 0.92 ms at 4k, 3.09 vs 3.09 ms at 8k, and 11.52 vs 11.52 ms at 16k. Both outperform naive JAX, which reaches 20.36 ms at 16k.

Flexible TPU Interface

A usable TPU interface must support not just forward execution, but also **training-time differentiation**. We therefore compared the backward pass of our flexible implementation against Splash on the same soft-cap workload.

- **Main result**: adding a flexible **score_mod** interface introduces essentially no backward overhead.
- This suggests the abstraction is practical for training, not just forward-only experiments.



Backward pass. Flex and Splash remain nearly identical at longer sequence lengths: 30.47 vs 30.38 ms at 16k, 120.10 vs 119.88 ms at 32k, and 471.30 vs 470.29 ms at 65k. The takeaway is that flexibility in the frontend does not noticeably degrade backward performance.

References

1. J. Dong, B. Feng, D. Guessous, Y. Liang, H. He, *FlexAttention: A Programming Model for Generating Optimized Attention Kernels*, 2024.
2. T. Dao et al., *FlashAttention*, 2022; T. Dao, *FlashAttention-2*, 2024.
3. O. Press, N. A. Smith, M. Lewis, *Train Short, Test Long: Attention with Linear Biases Enables Input Length Extrapolation*, 2022.
4. W. Kwon et al., *PagedAttention for LLM Serving*, 2023.
5. JAX documentation on Pallas and TPU kernel programming.